

Chapter 1

Introduction

A preview of Computer Programming

Chapter Targets

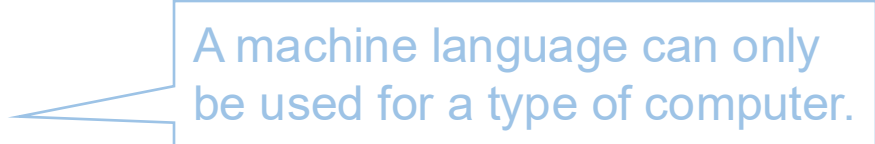
- The different types of programming languages
- The history of the C programming language
- The purpose of the C standard library
- A typical C program-development environment
- Get familiar with integrated development environment
- Leverage judge system to judge your program

Languages

- Three categories
 - Machine language
 - Assembly language
 - High-level language

Machine Language

- Consist of strings of numbers
 - Ultimately reduced to binary patterns
- Computer can understand only its machine language.
 - Machine dependent
- Cumbersome for human beings



A machine language can only be used for a type of computer.

Assembly Language

- English-like abbreviations to represent elementary operations
- Abbreviations as the basis of assembly language
- Assembler as translator program
 - Convert assembly language to machine language
- For instance,
 - load price
 - sub cost
 - store profit

High-level Languages

- Single statement could accomplish substantial tasks.
- **Compiler** as translator program to machine language
- More like the language we used everyday
- Mathematical notations are also available
- E.g.,
 - $\text{profit} = \text{price} - \text{cost}$

The C Programming Language

- Developed by Dennis Ritchie at Bell Lab. in 1972.
- Evolved from Basic Combined Programming Language (BCPL) and B.
- Hardware independent
 - Portable to most computers under careful design
- Built for performance
 - Widely used to develop operating systems, embedded systems, real-time systems, and communication systems

Standardization of C

- Extend C to various hardware platform leads to variations and **incompatibility**
- ANSI X3.159-1989 (ANSI C, Standard C, C89)
 - Approved in 1989 by ANSI
- ISO/IEC 9899:1990 (C90)
 - ANSI C standard adopted by ISO
- ISO/IEC 9899-1999 (C99)
- ISO/IEC 9899:2011 (C11)
- ISO/IEC 9899:2018 (C18)

American National
Standards Institute

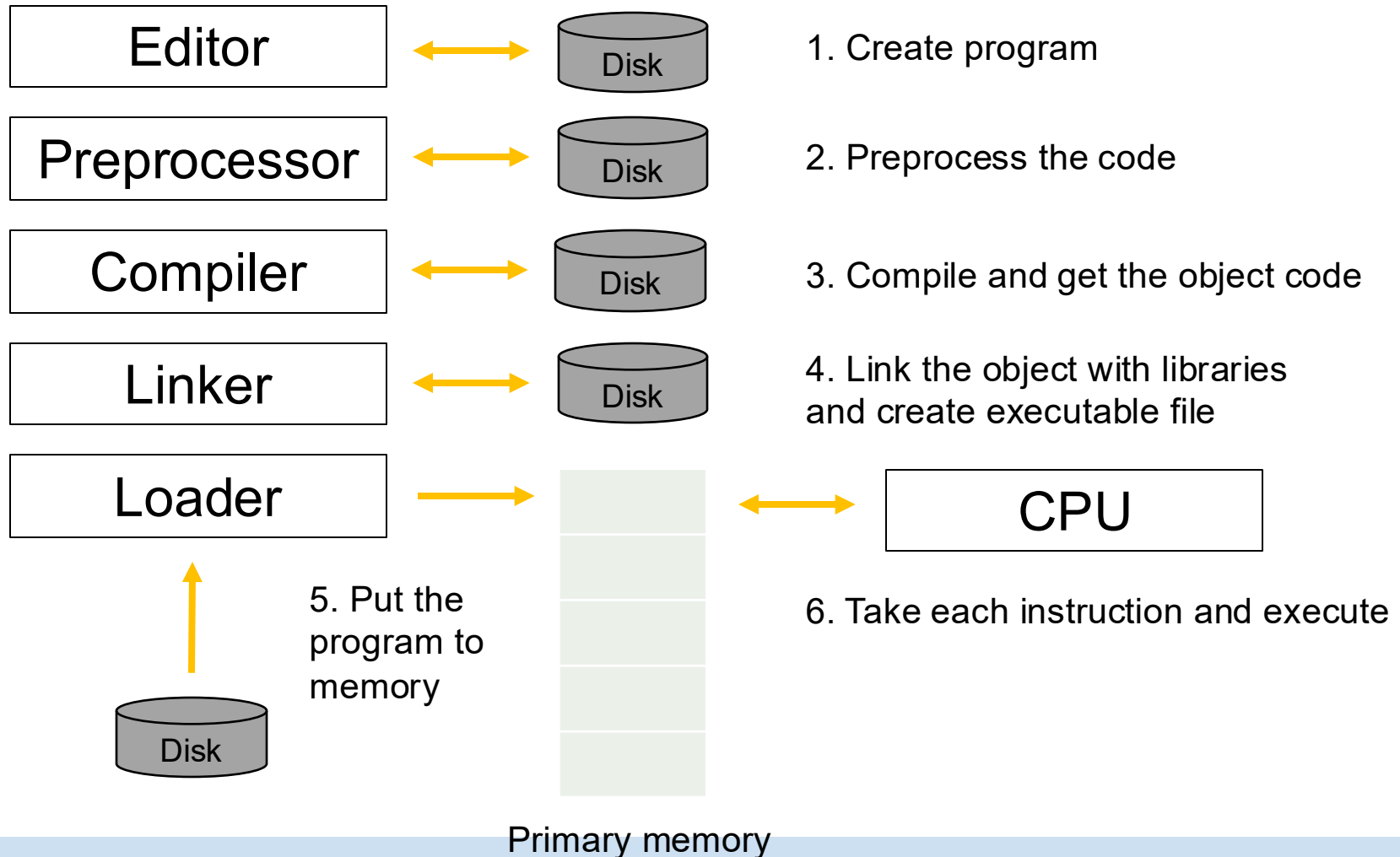
International Standards
Organization

International Electrotechnical Commission

C Standard Library

- A collection of useful functions
- How to program in C:
 - Learn the C language itself
 - Learn to use the functions in C Standard Library
- Software reuse
 - Avoid “reinventing the wheel”
- Building-block approach for programming
 - C Standard Library functions
 - Functions you create
 - Functions others create

Typical C Program-Development Environment



Typical C Program-Development Environment (cont'd)

1. **Create** program and store in `.c` extension such as `main.c`
2. **Preprocess**
 - Invoke automatically by preprocess according to the preprocessor directives, e.g. including other files in the file to be compiled, and performing various text replacements
3. **Compile**
 - Translate to machine-language code
 - Issue syntax error (compile error, compile-time error) if the code violates the rules of the language

Typical C Program-Development Environment (cont'd)

4. Link

- Program may contain references to functions elsewhere, and these parts are missing in object code.
- Linker links the code for the missing function, and produce executable image.
- Preprocess, compile, and link by:
`gcc main.c`

5. Load

- Loader transfers the executable image to memory for execution.
- Additional components from shared libraries are also transferred.

Typical C Program-Development Environment (cont'd)

6. Execute

- Finally, CPU executes the program one instruction at a time.
- Load and execute by:
./a.out
- Runtime error (execution-time error)
 - ◆ Error occurs while program runs.
 - ◆ Fatal error: The error makes the program terminate before completing the task.
 - ◆ Non-fatal error: The error allows the program to run to completion, but the result might be wrong.

Typical C Program-Development Environment.

- **Standard Input Stream** (`stdin`)
 - Certain C functions take their input from this stream, which is normally the **keyboard**.
- **Standard Output Stream** (`stdout`)
 - Programs often output data to this stream, which is normally the **screen**.
 - Such stream can be redirected to other devices such disk or printer.
- **Standard Error Stream** (`stderr`)
 - Program output the error message to such stream, which is normally the **screen**.
 - It is common to route the `stdout` to a device other than screen, and keep the `stderr` assigned to screen.

C Compiler

- Linux

- GNU Compiler Collection (GCC)

- Install `gcc` in the Linux based OS, for example the Ubuntu

- ```
sudo apt-get install gcc
```

- Windows

- Minimalist GNU for Windows (MinGW)

- Download from the official site

- <http://www.mingw.org>

- Install by the installation manager

- ◆ The default folder for MinGW is at `C:\MinGW`

# Environmental Variables

- The way of compiling your code is almost the same in Linux and Windows systems.
- In windows, we have to add the path of compiler to the system first.
  - Open the file manager (檔案總管)
  - Write click my computer (本機)
  - Left click content (內容) in the list
  - Left click the advanced system setting (進階系統設定)
  - Left click environmental variables (環境變數)
  - Add C:\MinGW\bin to “Path” in the user variables (使用者變數)
    - ◆ Left click Path, left click edit (編輯) and left click add (新增)

# Compile

- Use command line to compile your code
  1. Open the terminal (終端) in a Linux system or command line (命令提示字元) in a Windows system
  2. Change the directory to where you put your code
    - ◆ Use the command “cd” which means change directory
    - ◆ If your code is placed in different disk, use “X:” to change the disk, where X is the disk number.
    - ◆ For example, if your code is placed at D:\mycode\project1, then type the following commands to change to the directory:
      - D:
      - cd mycode\project1 or cd “D:\mycode\project1”

# Compile.

3. Invoke GCC in Linux or MinGW in Windows to compile your code by the command `gcc`
  - ◆ For example, compile you `main.c` file by typing:  
`gcc main.c`
  - ◆ The resultant file names are different in Linux and in Windows systems
    - In Linux, it is `a.out`
    - In Windows it is `a.exe`
  - ◆ We can also specify the file name of the executable by using the `-o` flag. For example, we may type
    - `gcc main.c -o main.exe` (for windows)
    - `gcc main.c -o main` (for Linux)

# Flags

- The compiler uses different flags to control the compilation parameter.
- Some common flags are listed below:
  - `-o filename`: specify the file name of the executable  
`gcc main.c -o main.exe`
  - `-std=standard`: specify the C standard  
`gcc main.c -std=c89`
  - `-Wall`: list all warnings  
`gcc main.c -Wall`
  - `-O1, -O2, -O3`: set optimization level  
`gcc main.c -O3`

# Flags (cont'd)

- We can also use multiple flags in a command:

```
gcc main.c -o main.exe -std=c89 -Wall
```

- The flags can be used in each IDE. For example, in the Code::Blocks:
  - Left click Settings
  - Left click Compiler
  - Left click other compiler options
  - Add flags in the blank area

# Flags.

- Some other useful flags are listed below:
  - -g: Debug mode
  - -E: Preprocess only

```
gcc main.c -E -o preprocessed_main.c
```
  - -S: Compile only
  - -C: Compile and assemble, but do not link
  - --version: Figure out the version of the compiler
  - --help: Provide useful information related to flags

# Redirect Standard Stream

- Use > to redirect the standard output stream to a file

```
main.exe > file.txt
```

- Use < to redirect the standard input stream from a file

```
main.exe < file.txt
```

- Combine the two redirections

```
main.exe < input.txt > output.txt
```

# Integrated Development Environment (IDE)

- Code::Blocks
  - <http://www.codeblocks.org/>
- Visual Studio Community
  - <https://visualstudio.microsoft.com/zh-hant/vs/community/>
- Visual Studio Code
  - <https://code.visualstudio.com/>
- Dev-C++
  - <https://sourceforge.net/projects/orwelldevcpp/>

# Test-Driving a C Application

## Various Ways

- We'll show how to compile and execute C code using:
  - Microsoft Visual Studio Community Edition for Windows
  - Code::Blocks on Windows
  - GNU gcc in a shell on Linux

# Microsoft Visual Studio Community Edition for Windows

- <https://visualstudio.microsoft.com/downloads/>
- Click **Free Download** under **Community**
  - Click **Run** to start the installation process
  - Otherwise, double-click the installer file in your **Downloads** folder
- In the **User Account Control** dialog, click **Yes**
  - Allows the installer to make changes to your system
- In the **Visual Studio Installer** dialog, click **Continue**
  - Allows the installer to download the components it needs
- Select the option **Desktop Development with C**
  - Includes the Visual C compiler and the C standard libraries
- Click **Install**

# Microsoft Visual Studio Community Edition for Windows

- Click Build New Project 建立新專案
- Double Click Console Application 主控台應用程式
- Set Project Title and Location
- Click  to build and run

# Code::Blocks on Windows

- Install and Download Code::Blocks without Compiler
- Install compiler for supporting the newest C standard
  - Download the compiler from the Code:Blocks forum <https://forums.codeblocks.org/>
    - ◆ Check the post of nightly build
  - Extract the zip file to a folder, e.g., C:\mingw64
  - Run command prompt and call gcc --version to check the gcc version

# Code::Blocks on Windows

- Set mingw-w64 as default compiler
  - Launch Code::Blocks
  - Click Settings -> Compiler
  - Select GNU GCC Compiler
  - Click Copy and set the name of compiler, e.g., MinGW-W64
  - Select Toolchain executables
  - Set Compiler's installation directory at C:\mingw64\bin
  - Click Set as default to change the default compiler

# Code::Blocks on Windows

- Create a new project
  - File -> New -> Project
  - Double-click Console application
  - Select C as the language
  - Set Project Title and Location
  - Select Compiler and Finish
  - Click  to build and run

# Code::Blocks on Windows

- Set Perspective

- View > Perspectives > Delete current
- In view, commonly toggle
  - ◆ Toolbars
  - ◆ Manager
  - ◆ Logs
  - ◆ Start page

# Code::Blocks on Windows

- Hotkeys

- Execution: F9
- Execution in debug mode: F8
- Comment: Ctrl+Shift+C
- Uncomment: Ctrl+Shift+X
- Duplicate the line of the cursor: Ctrl+D
- Indent: Tab
- Unindent: Shift-Tab
- Close a tab: Click with the middle mouse button

# GNU g++ in a shell on Linux

- Take ubuntu as an example
- Open an terminal
- Install gcc by the following commands
  - `sudo apt-get update`
  - `sudo apt-get install gcc-13 g++-13`
  - `sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-13 60`  
`--slave /usr/bin/g++ g++ /usr/bin/g++-13`
  - `gcc --version`

# GNU g++ in a shell on Linux

- Coding via text editor
- Compile and run the program
  - `gcc -std=c17 -Wall -o main main.cpp`
  - `./main`

# Judge System

- Set the Compiler Path

- Open file explorer (檔案總管)
- Right click my computer (本機)
- Left click content (內容)
- Left click advanced system setting (進階系統設定)
- Left click environmental variables (環境變數)
- Left click the variable "Path"
- Left click edit
- Check if the path where your compile is located is in the list
- If no, left click Add (新增), type or paste the path, and click ok (確定)

# Judge System

- Login

- Download the .zip file.
- Unzip it to a single folder.
- Click the judgeSystem.exe executable file.
- Enter username as your student ID.
- Enter password as announced.
- Now you may have entered the judge system.

# Judge System

- Judge

- Complete your source code
- Use the [list button](#) to select the problem where you want to judge
- Click [select file](#) to select all the source code required for the selected problem
- If your compiler by default is **not** at “C:\MinGW\bin\”, you have to [select your compiler](#).
  - ◆ Click the path and select your compiler (gcc.exe or g++.exe) manually
  - ◆ A possible path may be C:\Program Files (x86)\CodeBlocks\MinGW\bin\
  - ◆ This step must be taken once for each time you login to the system

# Judge System

- Judge

- Click [Proceed button](#) to judge your answer
  - ◆ If the code is correct, you may see Accepted.
  - ◆ If the code is incorrect, you may see
    - Compilation Error: your code cannot be compiled.
    - Wrong Answer: the answer is wrong. You may use the [Compare button](#) to see the differences between your results and the ground truth.
    - Run-time Error: the program has problem during run-time.
    - Time Limit Exceeded: the program is too inefficient to output the result.

Practice makes perfect.

# Exercises

- What are the default devices of standard input stream, standard output stream, and standard error stream?
- Give the command to compile the source file `my_file.c` to the executable `my_file.exe`.
- Show the typical C program-development environment.